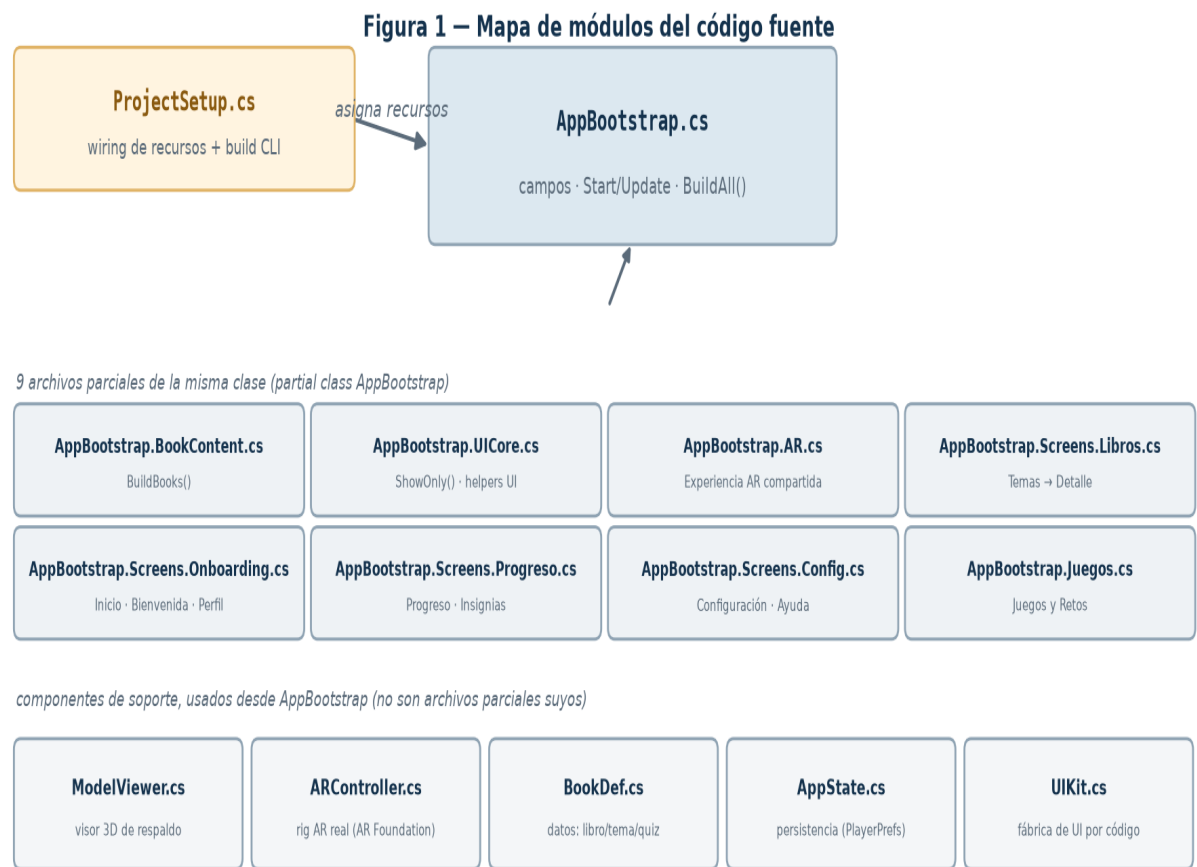


Aplicación ARDiabetes (Unity) — Arquitectura del código y gestión de modelos 3D

Proyecto: ARDiabetes — aplicación educativa de realidad aumentada sobre diabetes tipo 1
Repositorio: ~/UnityProjects/ARDiabetes
Motor: Unity 6 (6000.5.2f1), escena única, interfaz construida íntegramente por código
Fecha: 26 de julio de 2026

1. Arquitectura general del código fuente

El comportamiento de la aplicación se reparte en los siguientes módulos, cada uno con una función claramente delimitada, resumidos en la figura 1.



`AppBootstrap.cs` actúa como punto de entrada: declara los campos que reciben los recursos (sprites, modelos, audio, marcadores), gestiona el ciclo de vida de Unity (`Start` y `Update`) y coordina la construcción de las veintidós pantallas de la aplicación a través del método `BuildAll()`. El contenido educativo de los tres

libros —títulos, descripciones en tres niveles de lenguaje según la edad del usuario, preguntas de quiz y referencias a audio— se define íntegramente en `AppBootstrap.BookContent.cs`, dentro del método `BuildBooks()`. La navegación entre pantallas y la activación o desactivación del subsistema de realidad aumentada según la pantalla visible se resuelven en `AppBootstrap.UICore.cs`, en el método `ShowOnly()`, que constituye el punto central de orquestación de la aplicación. La pantalla de Experiencia AR, compartida por los tres libros y por el modo de escaneo genérico, junto con la narración por audio y la captura de fotografías, se implementa en `AppBootstrap.AR.cs`. El resto de las pantallas —Temas y Detalle de cada libro, Onboarding, Progreso, Configuración y Juegos— se distribuyen en sus respectivos archivos `AppBootstrap.Screens.*.cs` y `AppBootstrap.Juegos.cs`.

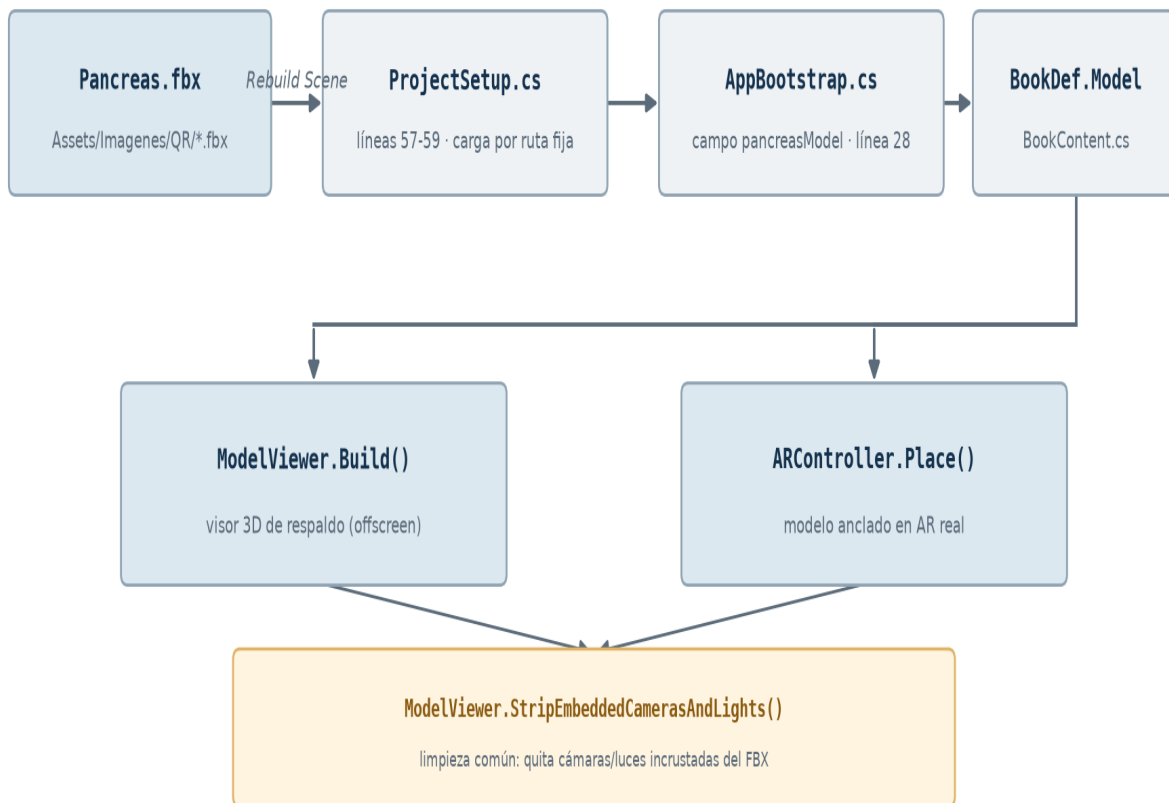
Fuera de la clase `AppBootstrap`, tres componentes son responsables de la representación tridimensional y la persistencia de datos: `ModelViewer.cs` implementa un visor 3D de respaldo que renderiza el modelo fuera de pantalla; `ARController.cs` implementa el rig de realidad aumentada propiamente dicho, basado en AR Foundation y ARCore; y `AppState.cs` gestiona la persistencia del progreso del usuario mediante `PlayerPrefs` (estrellas, nivel, racha diaria, temas vistos y perfil de edad). La clase `BookDef.cs`, por su parte, define exclusivamente la estructura de datos de un libro y de una pregunta de quiz, sin lógica de comportamiento. Finalmente, `UIKit.cs` funciona como una biblioteca interna de construcción de interfaz: genera en tiempo de ejecución los sprites necesarios (rectángulos redondeados, círculos, degradados) y provee funciones de fábrica para botones, textos y contenedores, posicionados mediante un sistema de fracciones que garantiza el comportamiento responsivo de la aplicación tanto en orientación vertical como horizontal.

Todo el cableado de recursos —qué archivo de imagen, modelo o audio corresponde a cada campo de `AppBootstrap`— se concentra en un único script de editor, `Assets/Editor/ProjectSetup.cs`, accesible desde el menú ARDiabetes de Unity. Este script también expone los métodos de compilación para Android y para escritorio Linux (este último utilizado como entorno de validación visual sin necesidad de un dispositivo físico), así como un submenú de vista previa que permite inspeccionar cualquiera de las veintidós pantallas de la aplicación directamente en el editor, sin ejecutar la aplicación completa.

2. Gestión de los modelos 3D

Dado que la correcta identificación de los modelos tridimensionales es un aspecto operativo central del proyecto, se detalla a continuación el recorrido completo de cada modelo, desde el archivo de origen hasta su renderizado en pantalla (figura 2).

Figura 2 — Recorrido de un modelo 3D, del archivo FBX a la pantalla



2.1. Origen de los archivos

Los tres modelos son archivos en formato FBX, exportados originalmente de plataformas como Sketchfab o Meshy, y residen en la carpeta `Assets/Imagenes/QR/` bajo los siguientes nombres: `Pancreas.fbx` (18,7 MB, correspondiente al Libro Fisiológico), `Plato.fbx` (27,7 MB, Libro Nutricional) y `Gluco.fbx` (28,6 MB, Libro Clínico). Esta ubicación, aunque su nombre de carpeta sugiere una relación con los códigos QR, es simplemente la carpeta heredada del prototipo original del que se extrajeron los modelos.

2.2. Asignación de los archivos a la aplicación

La vinculación entre estos archivos y la aplicación se realiza en `Assets/Editor/ProjectSetup.cs`, líneas 57 a 59, donde el método `Build()` carga cada archivo por su ruta absoluta dentro del proyecto mediante `AssetDatabase.LoadAssetAtPath<GameObject>()` y lo asigna a un campo público de `AppBootstrap`:

```
boot.pancreasModel =
AssetDatabase.LoadAssetAtPath<GameObject>("Assets/Imagenes/QR/Pancreas.fbx");

boot.platoModel    =
AssetDatabase.LoadAssetAtPath<GameObject>("Assets/Imagenes/QR/Plato.fbx");

boot.glucoModel    =
AssetDatabase.LoadAssetAtPath<GameObject>("Assets/Imagenes/QR/Gluco.fbx");
```

Estos campos —`pancreasModel`, `platoModel` y `glucoModel`— están declarados en `AppBootstrap.cs`, línea 28, como referencias de tipo `GameObject`. Esta operación ocurre exclusivamente cuando se ejecuta el elemento de menú **ARDiabetes ► Rebuild Scene** (o su equivalente por línea de comandos, `-executeMethod ProjectSetup.Build`), momento en el cual se reconstruye la escena completa desde cero y se vuelve a realizar todo el cableado de recursos.

A partir de estos tres campos, el método `BuildBooks()` en `AppBootstrap.BookContent.cs` construye las tres instancias de `BookDef` que representan a cada libro, asignando el modelo correspondiente a la propiedad `Model` de cada una, junto con un color de tinte (`ModelTint`) que se aplicará posteriormente como material del modelo instanciado.

2.3. Procedimiento para reemplazar un modelo

En consecuencia, existen tres vías para modificar el modelo tridimensional asociado a un libro, en orden creciente de flexibilidad. La primera y más directa consiste en sustituir el archivo FBX correspondiente en `Assets/Imágenes/QR/` conservando exactamente el mismo nombre, y ejecutar posteriormente **ARDiabetes ► Rebuild Scene**; de esta manera el nuevo modelo se carga automáticamente sin necesidad de modificar código. La segunda vía consiste en modificar la ruta especificada en `ProjectSetup.cs` (líneas 57 a 59) si se desea utilizar un archivo con otro nombre o ubicado en otra carpeta, seguido igualmente de una reconstrucción de la escena. La tercera vía, útil para pruebas rápidas sin recompilar scripts, consiste en seleccionar el objeto `AppBootstrap` dentro de la escena ya abierta en el editor y arrastrar manualmente un prefab distinto al campo correspondiente en el Inspector; esta asignación manual, sin embargo, se pierde la próxima vez que se ejecute **Rebuild Scene**, ya que dicho proceso sobrescribe los tres campos con los valores cargados desde las rutas fijas.

2.4. Instanciación y tratamiento en tiempo de ejecución

Cada modelo se instancia en dos contextos independientes, con lógicas de escalado ligeramente distintas pero un tratamiento común en varios aspectos.

El primer contexto es el **visor 3D de respaldo**, implementado en `ModelViewer.cs`. Al iniciar la aplicación (`AppBootstrap.cs`, línea 137), se crea una instancia de `ModelViewer` por cada uno de los tres libros mediante el método estático `ModelViewer.Create()`. Este visor instancia el modelo fuera del área visible de la escena, normaliza su escala a `Vector3.one` para eliminar cualquier escala no uniforme que el archivo FBX pudiera traer incorporada, calcula automáticamente sus límites geométricos (*bounds*) para centrarlo y escalarlo de manera uniforme hasta ocupar aproximadamente dos unidades de Unity, y lo renderiza mediante una cámara y una fuente de luz propias hacia una textura de renderizado (`RenderTexture`) de 800 por 900 píxeles, la cual se muestra finalmente en un elemento `RawImage` de la interfaz. Cada uno de los tres visores opera en una capa de renderizado distinta (capas 2, 3 y 4) para evitar que sus cámaras respectivas rendericen los tres modelos superpuestos, defecto que fue identificado y corregido durante el desarrollo. Adicionalmente, la cámara de cada visor permanece desactivada de forma predeterminada y solo se habilita cuando su libro correspondiente está efectivamente visible en pantalla, con el fin de evitar un costo de renderizado innecesario cuando existen tres visores simultáneos.

El segundo contexto es la **realidad aumentada real**, implementada en `ARController.cs`. Cuando la cámara del dispositivo detecta uno de los doce marcadores impresos (cuatro por libro), el método `Place()` instancia el modelo asociado a ese marcador —línea 261— y lo ancla como hijo del objeto `ARTrackedImage` que

representa la posición del marcador en el mundo real. Al igual que en el visor de respaldo, la escala se normaliza antes de aplicar un factor de autoescalado, en este caso calculado para que el modelo ocupe aproximadamente nueve centímetros físicos sobre el marcador. La rotación del modelo se gestiona de forma independiente mediante una variable acumulada (`userYaw`) que combina un giro automático continuo con la rotación manual que el usuario puede aplicar arrastrando el dedo sobre la pantalla.

Ambos contextos comparten dos operaciones críticas sobre el modelo instanciado. La primera es la eliminación de componentes de cámara, luz y `AudioListener` que el archivo FBX pudiera traer incorporados de su escena de origen, mediante el método estático y público `ModelViewer.StripEmbeddedCamerasAndLights()`; esta limpieza resultó indispensable durante el desarrollo, ya que una cámara incrustada en el modelo del páncreas competía silenciosamente con la cámara real de realidad aumentada y provocaba que el flujo de video apareciera cubierto por un fondo de cielo azul liso. La segunda operación común es la sustitución de todos los materiales del modelo por un material estándar del *render pipeline* Built-in, coloreado con el tinte asignado en `BookDef`, dado que los materiales originales del pipeline universal (URP) del prototipo se renderizan con una tonalidad rosada incorrecta bajo el pipeline actualmente en uso.

2.5. Resumen del recorrido de un modelo

Etapas	Ubicación
Archivo fuente (FBX)	<code>Assets/Imagenes/QR/Pancreas.fbx · Plato.fbx · Gluco.fbx</code>
Carga y asignación	<code>ProjectSetup.cs</code> (líneas 57-59) → campos <code>AppBootstrap.cs:28</code>
Entrada al contenido	<code>AppBootstrap.BookContent.cs</code> → <code>BookDef.Model</code>
Instanciación (visor 3D)	<code>ModelViewer.cs</code> → <code>Build()</code>
Instanciación (AR real)	<code>ARController.cs</code> → <code>Place()</code>
Limpieza común	<code>ModelViewer.StripEmbeddedCamerasAndLights()</code>

3. Marcadores de realidad aumentada

Los doce marcadores QR que activan la detección de cada tema —cuatro por libro— residen en `Assets/Markers/` y se cargan por nombre de archivo fijo en `ProjectSetup.cs`, líneas 95 a 97. Todos los marcadores de los tres libros se registran conjuntamente en un único `ARController` al iniciar la aplicación (`AppBootstrap.cs`, líneas 143 a 155), decisión arquitectónica motivada por una limitación de AR Foundation: el framework admite únicamente una sesión de `ARSession` activa por proceso, por lo que crear un controlador independiente por libro provocaba errores nativos de la cámara al alternar entre ellos. En su lugar, cada pantalla restringe en tiempo real el subconjunto de marcadores que puede reconocer mediante el método `SetScope()`, sin necesidad de crear ni destruir ninguna sesión de realidad aumentada.

El tamaño físico esperado de cada marcador está codificado de forma fija en `ARController.cs`, línea 235, con un valor de doce centímetros. Es importante que cualquier marcador impreso respete esta medida, dado que una discrepancia entre el tamaño declarado a `ARCore` y el tamaño físico real del impreso degrada la estabilidad del seguimiento (*tracking*) del modelo anclado.

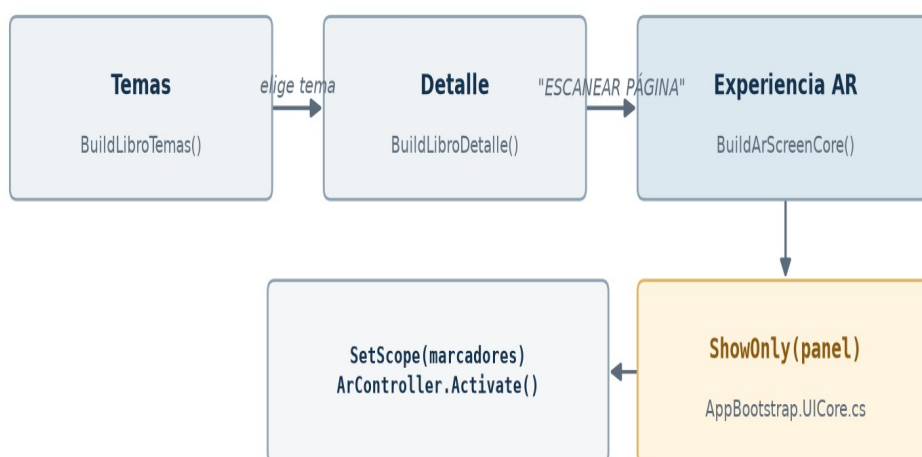
4. Contenido educativo

La totalidad del contenido pedagógico se define en `AppBootstrap.BookContent.cs`. Cada libro incluye cuatro temas con su título, subtítulo e ícono correspondiente, y cada tema cuenta con tres versiones de su descripción textual, redactadas en distintos niveles de lenguaje según el rango de edad del perfil seleccionado por el usuario (cinco a nueve años, diez a doce años, y trece a quince años). Esta diferenciación no es meramente cosmética: determina el texto efectivamente mostrado y narrado por audio. Cada libro incluye además tres preguntas de evaluación de tipos distintos —opción múltiple, relación de conceptos y selección múltiple—, y una referencia a los archivos de narración en formato WAV ubicados en `Assets/Audio/Narracion/`.

5. Flujo de navegación y activación de la realidad aumentada

El método `ShowOnly()`, en `AppBootstrap.UICore.cs` (líneas 182 a 261), constituye el punto de control central de la aplicación: determina qué panel debe mostrarse y ejecuta la lógica de transición correspondiente. Cuando el panel de destino corresponde a la Experiencia AR de un libro específico, se asigna el visor 3D de ese libro, se restringe el alcance del controlador de realidad aumentada a los marcadores de dicho libro y se activa la sesión de AR. Cuando el destino es el modo de escaneo genérico, el alcance se deja sin restricción, permitiendo la detección de cualquiera de los doce marcadores. Al abandonar cualquier pantalla de realidad aumentada, la sesión se desactiva y se restauran la cámara principal y el fondo decorativo de la interfaz.

Figura 3 – Navegación de un libro y activación de la sesión AR



Cabe destacar una salvaguarda implementada en `AppBootstrap.cs`, línea 203: la interfaz completa se reconstruye automáticamente cada vez que cambia la orientación del dispositivo, excepto mientras el usuario se encuentra dentro de una pantalla de realidad aumentada activa, condición en la cual dicha reconstrucción se suprime deliberadamente para evitar interrumpir el flujo de video de la cámara en pleno funcionamiento.

6. Compilación y despliegue

El proceso de generación de artefactos de instalación se realiza íntegramente por línea de comandos, sin requerir la interfaz gráfica de Unity. La compilación para Android se ejecuta mediante el comando siguiente, y el paquete resultante puede instalarse directamente en un dispositivo conectado mediante `adb install`:

```
Unity -batchmode -quit -projectPath ~/UnityProjects/ARDiabetes -buildTarget Android
-executeMethod ProjectSetup.BuildAndroid

adb install -r -g /tmp/ardiabetes_apk/ARDiabetes.apk
```

De forma complementaria, `ProjectSetup.BuildLinux()` genera un ejecutable de escritorio que permite validar visualmente el comportamiento de las pantallas sin necesidad de un dispositivo Android, recurso particularmente útil combinado con la rutina de captura automática (`AutoCapture()`, en `AppBootstrap.AR.cs`, línea 142), la cual recorre las veintidós pantallas de la aplicación y genera una captura de cada una.

7. Mapa de archivos de referencia

Archivo	Responsabilidad
AppBootstrap.cs	Campos, ciclo de vida, BuildAll() (arma las 22 pantallas)
AppBootstrap.BookContent.cs	BuildBooks() — contenido de los 3 libros
AppBootstrap.UICore.cs	ShowOnly() (navegación + activación de AR), helpers de UI reusables
AppBootstrap.AR.cs	Pantalla de Experiencia AR compartida, narración, captura de foto
AppBootstrap.Screens.Libros.cs	Pantallas Temas → Detalle de cada libro
AppBootstrap.Screens.Onboarding.cs	Inicio, Bienvenida (carrusel), Perfil
AppBootstrap.Screens.Progreso.cs	Progreso, Insignias, Colección
AppBootstrap.Screens.Config.cs	Configuración, Ayuda
AppBootstrap.Juegos.cs	Pantalla de Juegos y Retos
AppBootstrap.Refs.cs	ArRefs / DetalleRefs — referencias por panel construido
ModelViewer.cs	Visor 3D de respaldo offscreen → RenderTexture

Archivo	Responsabilidad
ARController.cs	Rig AR real: ARSession/XROrigin/ARCamera/ARTrackedImage Manager
BookDef.cs	Forma de datos: libro, tema, quiz — sin lógica
AppState.cs	Persistencia (PlayerPrefs): estrellas, nivel/XP, racha, perfil de edad
UIKit.cs	Fábrica de UI por código: paleta, sprites en runtime, layout por fracciones
Carousel.cs, Appear.cs, PressEffect.cs, SafeArea.cs, Spinner.cs	Componentes chicos de UI/interacción reutilizados
Assets/Editor/ProjectSetup.cs	Wiring de recursos, build Android/Linux por CLI, SetupAR(), menú de Preview